

0.96. Une lettre qui tire — un tir, tout seul

Le plus petit tir possible : une lettre A en (2, 8), et un « . » qui part juste à sa droite, avance tout seul d'une case toutes les 100 ms, et disparaît au bord de l'écran. Une seule variable, la colonne du tir.

1. Ce qu'il faut comprendre

Une nouvelle série : une lettre qui tire. Dans beaucoup de jeux, le héros **tire** : une petite chose part de lui, file tout droit, et touche ce qu'elle rencontre. On va le construire pas à pas. Ici, le plus petit morceau : **la lettre A, et un tir qui part d'elle**. Pas de bouton, pas de cible.

La lettre qui tire : un A, posé une fois en (2, 8) avec `poser(2, 8, ALPHABET[0])`. Il ne bouge pas.

Un tir, c'est quoi ? C'est un signe qui bouge, comme la lettre du 0.65 : on l'efface, on change sa colonne, on le redessine. Avec **deux différences** : il va **vite**, et au bord de l'écran, il **disparaît** au lieu de revenir à gauche.

Une seule variable : `tx`, la colonne du tir. Elle part de **3** : le A est en colonne 2, le tir commence **juste à sa droite**. Pas sur le A : il le cacherait, puis l'effacerait en avançant. Sa ligne ne change jamais : c'est la ligne 8, celle du A, écrite directement dans le code.

Le signe du tir : `texte(tx, 8, ". ")` écrit un mot d'un seul signe, le **point**. C'est le dessin de **tous** les tirs de cette série : ceux du A, dans tous les sens, et plus tard ceux du B.

Vite : `chaque(100)` (le 0.77) : un pas toutes les 100 ms, **10 cases par seconde**. Le A du 0.66 n'en faisait que 7 : un tir va plus vite que celui qui le lance.

Disparaître : l'écran a les colonnes 0 à 19. Tant que `tx < 20`, le tir avance : on efface sa case, `tx = tx + 1`, et **s'il est encore dans l'écran**, on le redessine. Quand `tx` arrive à **20**, on ne le redessine pas : il a disparu. Et comme `tx < 20` est faux pour toujours, le bloc ne fait plus rien.

Déroulons : le A en (2, 8), `tx = 3`, le « . » en (3, 8). 100 ms plus tard : on efface (3, 8), `tx = 4`, on dessine en (4, 8). ... `tx = 19` : on efface (19, 8), `tx = 20` : `20 < 20` est faux, on ne dessine rien. Sur la ligne 8, il ne reste que le A.

Essaie : rends le tir plus lent avec `chaque(300)`. Ou pose le A en (6, 8) : change aussi `tx` en 7, pour que le tir parte toujours juste à côté de lui.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **459 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l’a demandé — et ce qu’il a laissé, parce que rien ne l’appelle :

Gravé dans la cartouche :

démarrage	53 o	← la console, à l'allumage – la mise en route de la console
chaque_minuteur()	124 o	← main() – une fonction fournie par le langage
main()	98 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte – attend de pouvoir écrire une case
AdresseCase	10 o	← texte(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
CopierTuiles	40 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	32 o	← CopierTuiles – 2 lettres sur 43
","	1 o	← texte() – un texte gravé
","	1 o	← texte() – un texte gravé

Lettres gravées : A.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 2	36 o sur place, dans main()	+ AdresseCase (10 o), EcrireTexte (10 o)
effacer()	× 1	15 o sur place, dans main()	+ AdresseCase (10 o), EffacerCases (12 o)
poser()	× 1	6 o sur place, dans main()	
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()	

Tes fonctions appelées :

```
chaque_minuteur() x 1      10 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : le tir, tout seul ----
// La lettre A est posée en (2, 8). Juste à sa droite part un TIR :
// un point « . » qui avance tout seul, case par case, et qui
// DISPARAÎT quand il arrive au bord de l'écran.
// Pas encore de bouton : le tir part dès le début, une seule fois.
// ALPHABET[0] : le A (on compte les lettres à partir de 0 : A = 0, B = 1...).

// Une variable, c'est une boîte qui garde un nombre.
// uint8_t : elle garde un nombre de 0 à 255.

#include <poser>           // pose une tuile sur une case du fond
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
#include <texte>            // écrit un texte à l'écran
#include <chaque>           // répond 1 toutes les n millisecondes
#include <effacer>         // efface des cases, ou tout le fond
```

2. UNE VARIABLE — TX

[illegible]

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(2, 8, ALPHABET[0]); // la lettre A, celle qui tire : colonne 2, ligne 8.
                             // Elle ne bougera pas : on la pose une fois.

    // texte(colonne, ligne, "...") écrit un mot ; ici, un mot d'un seul signe.
    texte(tx, 8, ".");       // le tir, à sa place de départ : (3, 8)

    while (true) {           // la boucle du jeu : elle tourne sans fin
        image();             // attendre l'image suivante (60 par seconde)

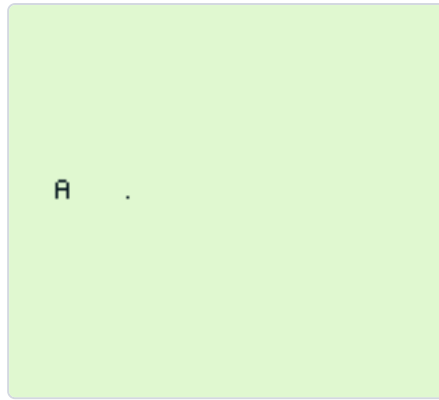
        // chaque(100) répond 1 toutes les 100 ms (le 0.77) :
        // le bloc se fait 10 fois par seconde. Un tir va VITE.
        if (chaque(100)) {
            if (tx < 20) {    // le tir est encore dans l'écran (colonnes 0 à 19) :
                effacer(tx, 8, 1); // 1. il quitte sa case (1 case effacée)...
                tx = tx + 1;      // 2. ...avance d'une colonne...
                if (tx < 20) {    // 3. ...et s'il est encore dans l'écran,
                    texte(tx, 8, "."); // on le dessine à sa nouvelle place.
                }
            }
            // tx vaut 20 : le tir est sorti par la droite. On ne dessine plus rien :
            // il a disparu, et « tx < 20 » est faux pour toujours : plus rien ne bouge.
            // Le A, lui, est toujours là.
        }
    }
}
```

3. Ce qui se passe, étape par étape

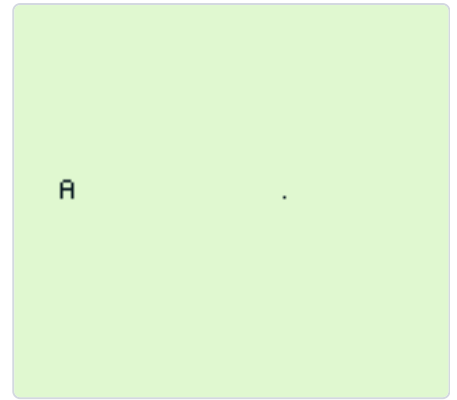
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



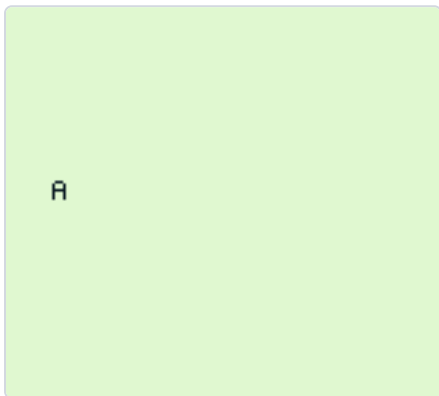
la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A en (2, 8). Juste à sa droite, un « . » file vers le bord, une case toutes les 100 ms, puis disparaît ; le A reste.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la lettre A est en (2, 8)
- ✓ le tir avance d'une colonne à la fois, vers la droite, sur la ligne 8 (colonnes 5 à 19)
- ✓ il va jusqu'à la colonne 19
- ✓ puis il disparaît : sur la ligne 8, il ne reste que le A

Ensuite : **0.96.1. Une lettre qui tire — un tir, tout seul, puis plusieurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++